MZ-80B Monitor SB-1520: English Companion Guide

Nov 7, 2026 · @Brett

An independent English guide to MONITOR SB-1520, the standard monitor program for the Sharp MZ-80B. It is written from scratch to accompany the scanned *Monitor Reference Manual SB-1520* (Sharp, June 1981, 080230-150681). It is not a translation. Facts about the code were checked against the monitor's assembly listing.

About the monitor

SB-1520 is a 4.5 KB program that sits at the bottom of RAM, from `$0000` to `$10E2`. It has two jobs.

- **Service library.** It holds the routines that drive the keyboard, screen, cassette deck, clock and sound. BASIC SB-5520, DISK BASIC SB-6520 / SB-6620, PASCAL and FDOS all call into it, and your own machine code can too.
- **Machine-language monitor.** You can hand control to it and use six one-letter commands to edit memory, dump memory, run code, and save, verify or load cassette files.

The monitor runs from RAM, so it can be changed: the restart vectors at `$0000`–`$0038` and the NMI vector at `$0066` can be repointed, and the monitor itself can be patched or saved to tape. Its work areas sit above it at `$1093`–`$12A0`, and BASIC SB-5520 starts at `$12A0`.

Address	What is there
`$0000`–`$003A`	Restart vectors (RST 0–7) interleaved with system variables
`$003B`	`START`: cold start after loading
`$00B1`	`ST`: the monitor command prompt `*`
`$1093`	`BUFER`: 80-byte line input buffer
`$1140`–`$11BF`	`IBUFE`: 128-byte cassette header buffer
`$11C0`–`$11FF`	Screen, key and sound work variables
`$12A0`	Start of BASIC SB-5520 (cold start `$12A0`, warm start `$1300`)

Address	What is there
`$D000`	Video RAM (`SCRN`)

Getting to the prompt. From BASIC SB-5520 type `MON`. PASCAL uses `Q/` and the assembler uses `!`. The monitor shows a `*` prompt and waits for a command.

Monitor commands

At the `*` prompt, type one letter and the monitor asks for what it needs, one prompt at a time.

Key	Command	Prompts	Result
`M`	Memory edit	`M-ADR.$`	Shows each byte from that address and lets you change it
`D`	Memory dump	`S-ADR.$`, `E-ADR.$`	Lists the block, 8 bytes per line
`J`	Jump	`J-ADR.$`	Runs code at that address
`S`	Save to tape	`FILE NAME:`, `S-ADR.$`, `E-ADR.$`, `J-ADR.$`	Writes the block to cassette
`V`	Verify tape	`FILE NAME:`	Compares a tape file with memory
`L`	Load from tape	`FILE NAME:`	Loads a file, then runs it if it has a jump address

Rules that apply to every command

- Everything is hexadecimal. Bytes take exactly 2 digits and addresses exactly 4, with leading zeros: type `0A`, not `A`.
- Press CR to enter a value. Stray characters such as spaces make the monitor reject the entry.
- BREAK abandons the command and returns to `*`.
- Nothing is protected. You can read, change or overwrite the monitor itself, which is useful but can crash the machine.

M: edit memory

Enter a start address. The monitor shows the address and its current byte, for example `70A0 00`. Type two hex digits and CR to replace the byte, or CR alone to keep it; either way it moves on to the next address. Typing a non-hex key brings back the `M-ADR.$` prompt, so you can jump to a new address. It is good practice to re-read what you entered, with M or D, before running it.

D: dump memory

Enter a start and an end address. The block is listed as an address followed by 8 bytes per line. Hold the space bar to pause the listing.

J: run code

Enter an address and the CPU jumps there; what happens next is up to that code. Two uses are worth knowing:

- **Breakpoints.** The byte `FF` is `RST 7`, which the monitor points at its register display (`REGIST`, `$0D07`). Put `FF` where you want to stop. When it runs, the monitor shows AF, BC, DE, HL and the address, then returns to `*`.
- **Returning to BASIC.** If BASIC SB-5520 is still intact in memory, `J` to `$1300` resumes it with its program and variables kept (hot start). `J` to `$12A0` restarts it from scratch (cold start).

S: save to tape

Give a file name of up to 16 characters, then the start and end addresses of the block, then an optional jump address (CR for none). Saving starts as soon as you press CR. As with BASIC's SAVE, the monitor shows `SET TAPE` if no cassette is loaded and `WRITE PROTECT` if the tab is out.

- A jump address makes the file run automatically after an `L` load. Leave it blank for routines that BASIC will call.
- Don't save a block that includes the monitor's own work areas while it is running. The data changes as it is written, so the checksum won't match on reload. Copy the block somewhere else first and save the copy.
- The end address is not saved. See **Known issue** below.

V: verify tape

Rewind, then give the file name (CR alone takes the first file found). The monitor compares the file with memory at the address recorded in the file, and reports whether they match. Run V straight after every S.

L: load from tape

Give the file name. The monitor shows `FOUND` with each file name it passes, then `LOADING` when it reaches yours. The file loads back to the address it was saved from. If it was saved with a jump address the program then starts; otherwise you return to `*`.

Cassette file format

Every tape file is a 128-byte header block followed by a data block. The monitor builds the header in `IBUFE` (`$1140`) before saving and reads it back there when loading. This section is worked out from the listing.

Offset	Address	Label	Size	Contents
`+00`	`$1140`	`ATRB`	1	File type; S writes `01` (machine code)
`+01`	`$1141`	`NAME`	17	File name, up to 16 characters, ended by `0D`
`+12`	`$1152`	`SIZE`	2	Data length in bytes
`+14`	`$1154`	`DTADR`	2	Load address
`+16`	`$1156`	`EXADR`	2	Jump (execution) address
`+18`	`$1158`	`COMNT`	104	Comment area, to the end of the header

How a block is recorded (`?WRI`, `?WRD`, `WTAPE`, `WBYTE`)

1. A lead-in of short pulses, then a tape mark of long and short pulses. The header gets 10,000 short, 40 long, 40 short, 1 long. The data block gets 11,000 short, 20 long, 20 short, 1 long.
2. The block, byte by byte. Each byte is a long start pulse, then 8 bits from the most significant down, with a long pulse for 1 and a short pulse for 0.
3. A 2-byte checksum, high byte first. It is a count of the 1 bits in the block (`CKSUM`).
4. The block and checksum are then written **a second time**, after 256 short pulses. Writing twice gives the loader a second chance if the first copy has a dropout. Changing `LD E,2` at `$02DA` to `LD E,1` saves a single copy.

Loading reads the header first, and takes the length and load address for the data block from it. `V` reads the same two blocks but compares instead of storing.

Subroutine reference

Call these from machine code with `CALL nnnn`, or from BASIC with `USR($nnnn)`. For example, `CALL $0A2E` (bytes `CD 2E 0A`) starts a new line on the screen. Every address below matches the entry label in the listing. Unless a routine says it preserves registers, assume any register it uses may change. Stack is the maximum depth used, in 2-byte levels, so reserve twice that many bytes.

Address	Name	What it does	In → out	Stack
`$0A2E`	`LETNL`	New line: cursor to the start of the next line	Keeps all but AF	8
`$08C4`	`PRNTS`	Print one space	—	3
`$08C6`	`PRNT`	Print the character in A; codes `$00`–`$1D` act as screen controls (`$01` = cursor down, and so on)	A = code	3
`$0889`	`MSG`	Print a string at the cursor	DE = text ending in `$0D` (not printed); keeps all registers	4
`$0F14`	`BELL`	Short beep on the note A, about 440 Hz	—	4
`$0F3F`	`MELDY`	Play a tune in BASIC `MUSIC` notation	DE = text ending in `$0D` or `*` → Carry set if BREAK stopped it	—
`$0E50`	`XTEMP`	Set tempo	A = `$01` (slowest) to `$07` (fastest); `$04` is medium	3
`$0F22`	`SOUT`	Play one tone	HL = pitch (`$00A4` = A), BC = duration	3
`$0E5E`	`TIMST`	Set the clock	A = 0 (AM) or 1 (PM), DE = seconds	3
`$0EA9`	`TIMRD`	Read the clock	→ A = 0/1 (AM/PM), DE = seconds	3
`$0562`	`BRKEY`	Test the BREAK key	→ Z set if pressed	2
`$06A4`	`GETL`	Read a line from the keyboard with echo and editing	DE = buffer; max length (including the `$0D` added at the end) in `$06A2`; BREAK stores `$0B` at DE	8

Address	Name	What it does	In → out	Stack
`$0832`	`GETKY`	Read the key held now, no echo, no debounce	→ A = ASCII code, or `$00` if none	8
`$05F3`	`ASC`	Low 4 bits of A → ASCII hex digit	A → A (`$08` → `$38`)	1
`$05FD`	`HEX`	ASCII hex digit → value	A → A low 4 bits; Carry set if not a hex digit	1
`$0623`	`2HEX`	Two ASCII hex digits → byte	DE = text → A; Carry set on error	2
`$0614`	`HLHEX`	Four ASCII hex digits → word	DE = text → HL (`"30C9"` → `$30C9`); Carry set on error	4
`$0C29`	`?PONT`	Cursor address in video RAM	→ HL (`$D000` = top-left)	2

Cassette entry points (from the listing). The manual's table doesn't list these, but they are the routines S, L and V use. Each uses the header in `IBUFE` (`$1140`) and returns with Carry set on error or BREAK.

Address	Label	What it does
`$0251`	`?WRI` (`WRINF`)	Write the 128-byte header
`$0282`	`?WRD` (`WRDAT`)	Write `SIZE` bytes from `DTADR`
`$028E`	`?RDI` (`RDINF`)	Read the next header into `IBUFE`
`$02B2`	`?RDD` (`RDDAT`)	Read `SIZE` bytes to `DTADR`
`$02BE`	`?VRFY` (`VERFY`)	Compare the data block on tape with memory

Known issue: SAVE writes one byte too few

The `S` command saves from the start address up to the byte **before** the end address. Saving `$6000` – `$60A3` stores `$A3` bytes, and the byte at `$60A3` is lost. The cause is at `$0196` : the monitor stores *end − start* as the length, not *end − start + 1*. If start and end are the same, the length is 0, and the write loop wraps round and tries to write 64 KB.

Workaround: give the end address as the last byte **+ 1**.

Fixes, from the listing and untested:

- Insert `INC HL` between `$0198` and `$019A`, as handwritten in one surviving copy of the manual. This needs the monitor reassembled, because every later address moves by one byte.
- Or change the single byte at `$0195` from `AF` (`XOR A`) to `23` (`INC HL`). The carry flag is already clear there, so `XOR A` isn't needed, and no other address changes.

What else is in the manual

The manual's own page numbers are given below. Apart from the commands and subroutines covered above, the rest is reference tables and code.

Pages	Section	Contents
1–16	Chapter 1.1–1.2	What the monitor does, and the six commands, with screen photos
17–20	Chapter 1.3	The subroutine table summarised above
22–31	Appendix A.1	Z80A instructions in alphabetical order of mnemonic, with their opcodes
32–41	Appendix A.2	The same instructions in opcode order, for working back from hex
42–86	Appendix A.3	The complete assembly listing of SB-1520, dated 31 January 1981

The Z80 tables are standard instruction-set data, so any Z80 reference covers the same ground. For timings and flag effects, the manual points to the Z80A technical data in the MZ-80B *Owner's Manual*.

The listing in A.3 is available as a checked text transcription. Each column is described on p. 42: address, object code, assembler error flag, label, mnemonic, operand and comment. The monitor is assembled at `$0000`, so the listed addresses are the real ones.

Sources

- *Personal Computer MZ-80B Monitor Reference Manual*, MONITOR SB-1520, Sharp Corporation, June 1981, part no. 080230-150681 (Japanese). Scanned copy, pp. 1–20 and 42.
- MONITOR SB-1520 assembly listing (Appendix A.3), transcribed to text and checked by disassembling every line. All addresses, labels and code behaviour in this guide were confirmed against it.

- Handwritten notes in the scanned copy, pp. 46–49, for the SAVE length issue.